

switch-uttryck

C++ har en konstruktion som är mycket praktisk då man vill välja en av flera olika alternativ. Om vi t.ex. har en meny där användaren kan välja alternativen 1, 2, 3 eller 4 kan vi implemnera det hela som en `if`-konstruktion, men det är ganska klumpigt och väldigt fult. Istället kan man använda `switch`-uttryck. Ett exempel illustrerar vad det är frågan om:

```
// läs ett menyval från tangentbordet
char Menyval;
cin >> Menyval;
switch ( Menyval ) {
    case '1' : cout << "1 valdes" << endl; break;
    case '2' : cout << "2 valdes" << endl; break;
    case '3' : cout << "3 valdes" << endl; break;
    case '4' : cout << "4 valdes" << endl; break;
}
```

Ovan har vi först en sats med nyckelordet `switch` som börjar `switch`-blocket. Denna tar som argument ett uttryck som skall evelueras till ett *heltal*. Observera att det skall vara ett heltal, inte ett boolskt uttryck som i `if`-satser. På bas av detta heltal kontrolleras varje `case`-sats för att se ifall värdet är samma. Om så är utförs `case`-satsens kropp. Så fortsätter evalueringen nedåt i `switch`-konstruktionen ända till slutet nåtts utan 'träffar', eller en eller flera `case`-satser utförts. Observera att syntaxen där ett tecken sätts innanför " ju ger tecknets ASCII-värde, d.v.s. ett tal. Den allmänna formen för en `case`-konstruktion är:

```
switch ( evaluating ) {
    case värde1 : sats1;
    case värde2 : sats2;
    ...
    case värdeN : satsN;
    default: sats;
}
```

Värdena *värde1* till *värdeN* måste vara distinkta, d.v.s. inga kopior tillåts förekomma. Den sista satsen `default` utförs normalt om ingen av de ovanstående `case`-satserna utfördes, och fungerar som en form av uppsamlare för alla värden som inte testats. Man kan se på `default` som den sista `else`-delen i en konstruktion med många `if`-satser.

Koden i en `case`-kropp är exceptionell på det viset att man inte där behöver gruppera koden i block. Satser utförs nedåt ända tills slutet av `switch`-konstruktionen nåtts, eller en `break` påträffats. Så följande exempel fungerar helt korrekt:

```
char Menyval;
cin >> Menyval;
switch ( Menyval ) {
    case '1' : cout << "1 valdes" << endl;
               Index++;
               cout << "Index är nu " << Index << endl;
               break;
    case '2' : ...
    ...
}
```

Här kommer alla satser att utföras om `Menyval == '1'`. Låter det ologiskt kanske? Varför måste inte `switch` ha en blockkonstruktion när allting annat måste ha det? Svaret är att `switch` använder sig av en ganska praktisk funktionalitet som kallas *fall through*. Det betyder att så fort som en `case`-test är sann kommer dess sats(er) att utföras. Exekveringen kommer dock efter att `case`-kroppen utförts att direkt fortsätta med kroppen för nästa `case`-sats, d.v.s. exekveringen "faller igenom". För att stoppa detta fenomen kan man använda sig av `break`. Denna stoppar exekveringen av `switch`-konstruktionen direkt och fortsätter på satsen *efter* själv `switch`-blocket. Ett exempel då *fall-through* är praktiskt är då man vill göra en liten meny som accepterar både versaler och gemener (stora och små bokstäver). Istället för att duplicera koden för båda fallen låter man den ena falla igenom:

```
char Menyval;
cin >> Menyval;
switch ( Menyval ) {
    case 'S' :
    case 's' : // spara en fil
                break;

    case 'l' :
    case 'L' : // ladda in en fil
                break;

    case 'q' :
    case 'Q' : // avsluta
                break;

    ...
    default : // felaktigt val!
                cout << "Felaktigt val!" << endl;
}
```

Här utnyttjas *fall through* på ett effektivt (och vackert!) sätt. I de flesta fall vill man dock inte ha denna effekt, så glöm inte bort att sätta en `break` där du inte vill ha *fall through*! Sist i konstruktionen ovan finns en `default`. Denna samlar upp alla värden på `Menyval` som inte matchades av någon `case`-sats. I vårt exempel har användaren då gett in ett menyval som inte kändes igen. Eftersom `break` används på alla ställen i de övriga `case`-satserna kan vi vara säkra på att alla giltiga val fångats upp och behandlats.

[Förutgående](#)
Logiska operatorer

[Hem](#)
[Upp](#)

[Nästa](#)
? :-operatorn